



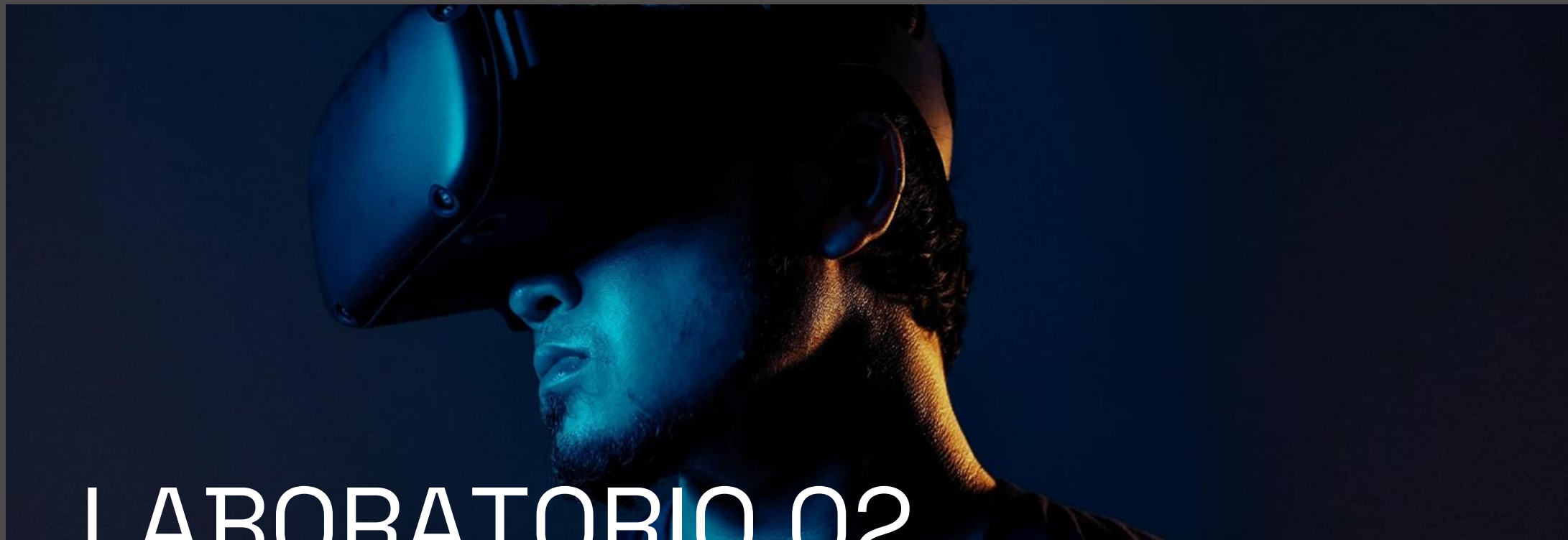
UTEC Posgrado

CURSO

INGENIERÍA DE DATOS

Mg. Angel Tintaya
Senior Data Engineer





LABORATORIO 02



Git

Sistema de control de versiones de código



Airflow

Herramienta para automatizar procesos de datos.



Docker

Herramienta que permite empaquetar y ejecutar aplicaciones en entornos aislados.





Git

Control de Versiones con GIT

- Git es un sistema de control de versiones de nuestro código.
- Nos permite trabajar de manera colaborativa sin perder el código avanzado.
- Todo proyecto se encuentra almacenado en un repositorio



Comandos básicos en GIT

Comando	¿Qué hace?
git init	Inicializar un repositorio
git status	Mostrar estados actuales
git add .	Preparar archivos para poder hacerles commit
git commit -m “...”	Guardar los cambios





Gitflow

GitFlow = Colaboración Organizada

Gitflow es un modelo de ramas que estructuran el trabajo.

Ramas comunes:

- **main**: Código listo para producción
- **develop**: Último desarrollo
- **feature/***: Nuevos cambios
- **hotfix/***: Cambios urgentes

Cada desarrollador trabaja en su propia rama





Gitflow - Pasos

Pasos	Descripción
Cambiar a una nueva rama	<code>git checkout -b feature/my-code</code>
Desarrollar el código y prepararlos para hacerles commit	<code>git add .</code>
Guardar cambios	<code>git commit -m "Add feature"</code>
Llevarlos a github	<code>git push origin feature/my-code</code>
Abrir un Pull Request (PR) a develop para su revisión y carga	

Practiquemos Gitflow



Apache Airflow



Qué es Apache Airflow?

- Airflow es una herramienta para automatizar procesos de datos.
- Organiza tareas en pipelines llamados DAGs.
- Muy usado en proyectos de ciencia de datos, ingeniería de datos y ML.

Qué es DAG?

- DAG =Directed Acyclic Graph (Grafo Acíclico Dirigido)
- Define qué tareas ejecutar y en qué orden.
- Cada tarea puede ser un script en python, una carga en la nube, etc.

Airflow ejecuta cada paso de forma ordenada y programada



Docker



Qué es Docker?

- Docker es una herramienta que permite empaquetar y ejecutar aplicaciones en entornos aislados llamados contenedores.
- Esto facilita la portabilidad, ya que la aplicación se ejecuta igual en cualquier máquina, sin importar el sistema operativo o las dependencias.



Comandos básicos en Docker con Airflow

Comando	¿Qué hace?
<code>docker compose up airflow-init</code>	Inicializa Airflow (base de datos, usuarios, etc.). Solo se corre una vez.
<code>docker compose up</code>	Inicia los servicios de Airflow en primer plano (muestra logs en consola).
<code>docker compose up -d</code>	Inicia Airflow en segundo plano (modo recomendado para trabajar).
<code>docker compose down && docker compose up -d</code>	Reinicia los servicios de Airflow (útil si algo se rompe).
<code>docker compose down && docker compose up --build -d</code>	Reinicia reconstruyendo las imágenes de Docker (solo si cambia el Dockerfile).



Ejecutemos Docker con Airflow



A hand is shown in the lower foreground, pointing its index finger towards a glowing network diagram on the left. The diagram consists of white nodes connected by lines, set against a dark blue background with a bright blue glow emanating from the nodes. The text is centered in the middle of the image.

Orquestación de Flujo de datos con Git y Apache Airflow

Gitflow + VS Code + Airflow

Objetivo

Desarrollar su primer DAG en Apache Airflow, trabajando en equipo y usando Gitflow para colaborar como lo harían en un entorno profesional.

¿Qué implica este mini-proyecto?

Crear un DAG personalizado que sube un archivo a la nube (Azure).

Usar buenas prácticas de desarrollo colaborativo:

- Ramas por funcionalidad
- Pull Requests
- Revisión de código



Gitlflow + VS Code + Airflow

Entregable por cada grupo:

Un archivo DAG en la carpeta dags/

Un archivo de prueba (raw/G3/archivo_subido.txt) subido a la nube

Un Pull Request en GitHub hacia la rama develop



Pasos

Pasos	Descripción	Código
Haz pull de la rama develop	Actualiza tu código local con lo último del equipo	git checkout develop git pull origin develop
Crea tu propia rama de trabajo	Usa un nombre único para evitar conflictos: Feature/dag-[Grupo]-[Iniciales]	git checkout -b feature/dag-g3-ATM
Crea tu archivo de DAG	Carpeta: dags/ Nombre del archivo: [Grupo]_[Iniciales].py Basado en el template entregado, actualiza la variable:	BLOB_NAME = "raw/G3/archivo_subido.txt"
Archivo a subir	Utilizaremos el archivo de ejemplo ubicado en: data/sample.txt	
Guarda, haz commit y push		git add dags/g3_ATM.py archivo_subido.txt git commit -m "DAG del grupo 3 - ATM" git push origin feature/dag-g3-ATM
Abre un Pull Request en GitHub	De tu rama → develop Título: DAG Grupo 3 – ATM Descripción: ¿Qué hace el DAG?	

Recomendaciones

- Antes de empezar el proyecto, les recomiendo practicar haciendo un cambio en el repositorio que crearon en clase.
- Se ha creado debajo de la carpeta dags el **archivo g0_atm.py**.
Tómenlo como guía, ya que tiene los principales cambios que deben aplicar
- **IMPORTANTE:** Cuando repliquen el DAG, acuérdense de **cambiar el nombre del dag_id**. Si airflow tiene dos dag_id iguales, sólo cargará el primero.
- Al levantar docker compose, recuerden agregar el “-d” en
“docker compose up --build -d”.

El parámetro -d significa detached mode, es decir, los contenedores se ejecutarán en segundo plano y podrán seguir usando el mismo terminal normalmente.

- Si es que no le pusieron el “-d”, lo que deben hacer es abrir otro terminal y ejecutar:
“docker compose down && docker compose up --build -d”
- **El proyecto termina sólo creando el PULL REQUEST**, no continuar haciendo el MERGE.
- Si ya terminaron, recuerden apagar el servicio con **“docker compose down”**

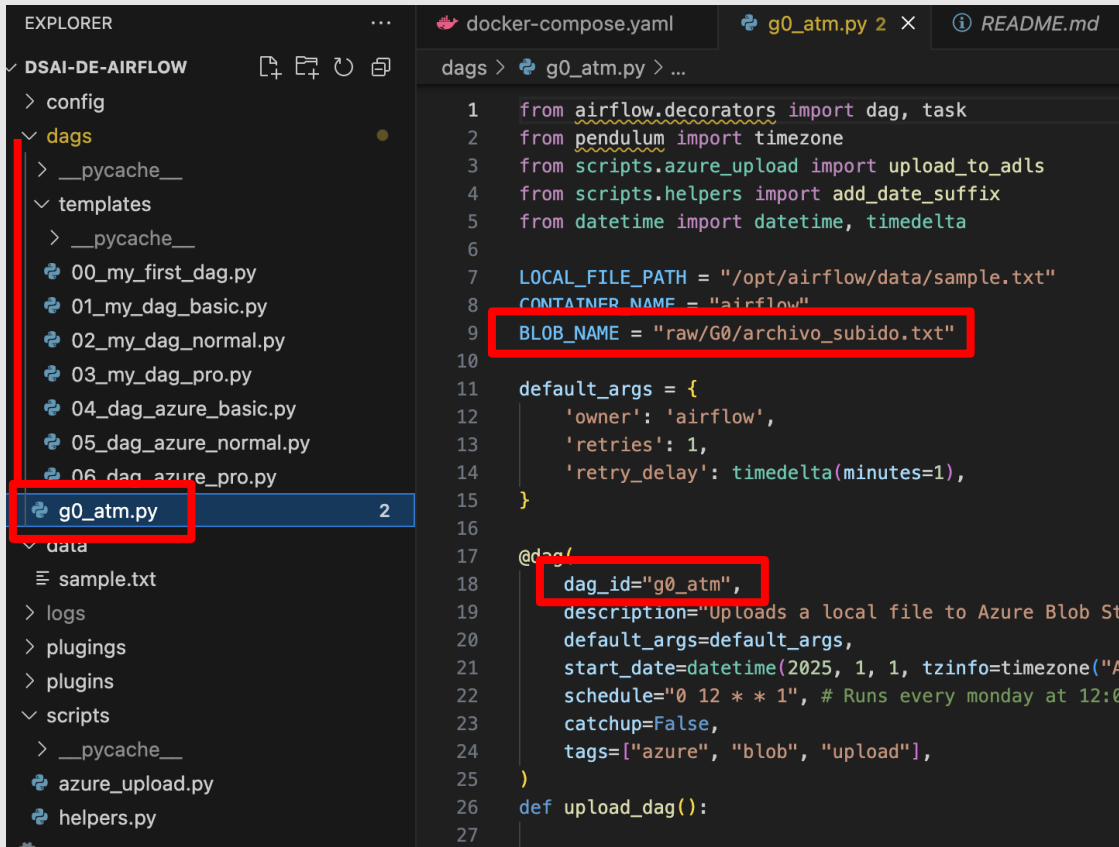
Como entregable, subir un pantallazo del código con el nombre del branch (Mayor detalle en siguiente diapositiva).

Se revisará que se haya creado el Pull Request y se tengan los cambios de la foto.



Ejemplo de implementación de cambios:

Se creó el
archivo
debajo de
dags, NO
templates



```

EXPLORER
DSAI-DE-AIRFLOW
├── config
├── dags
├── __pycache__
├── templates
├── __pycache__
├── 00_my_first_dag.py
├── 01_my_dag_basic.py
├── 02_my_dag_normal.py
├── 03_my_dag_pro.py
├── 04_dag_azure_basic.py
├── 05_dag_azure_normal.py
├── 06_dag_azure_pro.py
├── g0_atm.py
├── data
├── sample.txt
├── logs
├── plugings
├── plugins
├── scripts
├── __pycache__
├── azure_upload.py
├── helpers.py
└── ...

dags > g0_atm.py > ...
1 from airflow.decorators import dag, task
2 from pendulum import timezone
3 from scripts.azure_upload import upload_to_adls
4 from scripts.helpers import add_date_suffix
5 from datetime import datetime, timedelta
6
7 LOCAL_FILE_PATH = "/opt/airflow/data/sample.txt"
8 CONTAINER_NAME = "airflow"
9 BLOB_NAME = "raw/G0/archivo_subido.txt"
10
11 default_args = {
12     'owner': 'airflow',
13     'retries': 1,
14     'retry_delay': timedelta(minutes=1),
15 }
16
17 @dag(
18     dag_id="g0_atm",
19     description="Uploads a local file to Azure Blob Storage",
20     default_args=default_args,
21     start_date=datetime(2025, 1, 1, tzinfo=timezone("America/Mexico_City")),
22     schedule="0 12 * * 1", # Runs every monday at 12:00
23     catchup=False,
24     tags=["azure", "blob", "upload"],
25 )
26 def upload_dag():
27

```

Se actualizó el
BLOB_NAME

Importante!
Se actualizó el
nuevo dag_id



UTEC Posgrado